Machine Learning Lab3 Block1

Nazli Bilgic(nazbi056), Anastasia Piadi (anapi501), Max Björklund (maxbj107)

2023-12-18

Statement of Contribution

Anastasia (anapi501) was responsible for coding and writing analysis for assignment one.

Max (maxbj107) was responsible for coding and writing analysis for assignment two.

Nazli (nazbi056) was responsible for coding and writing analysis for assignment three.

Finalizing the last details of the report were done together.

1.Kernel Methods

Implement a kernel method to predict the hourly temperatures for a date and place in Sweden. To do so, you are provided with the files stations.csv and temps50k.csv. These files contain information about weather stations and temperature measurements in the stationsat different days and times. The data have been kindly provided by the Swedish Meteorologicaland Hydrological Institute (SMHI).You are asked to provide a temperature forecast for a date and place in Sweden. The forecast should consist of the predicted temperatures from 4 am to 24 pm in an interval of 2hours. Use a kernel that is the sum of three Gaussian kernels: The first to account for the physical distance from a station to the point of interest.

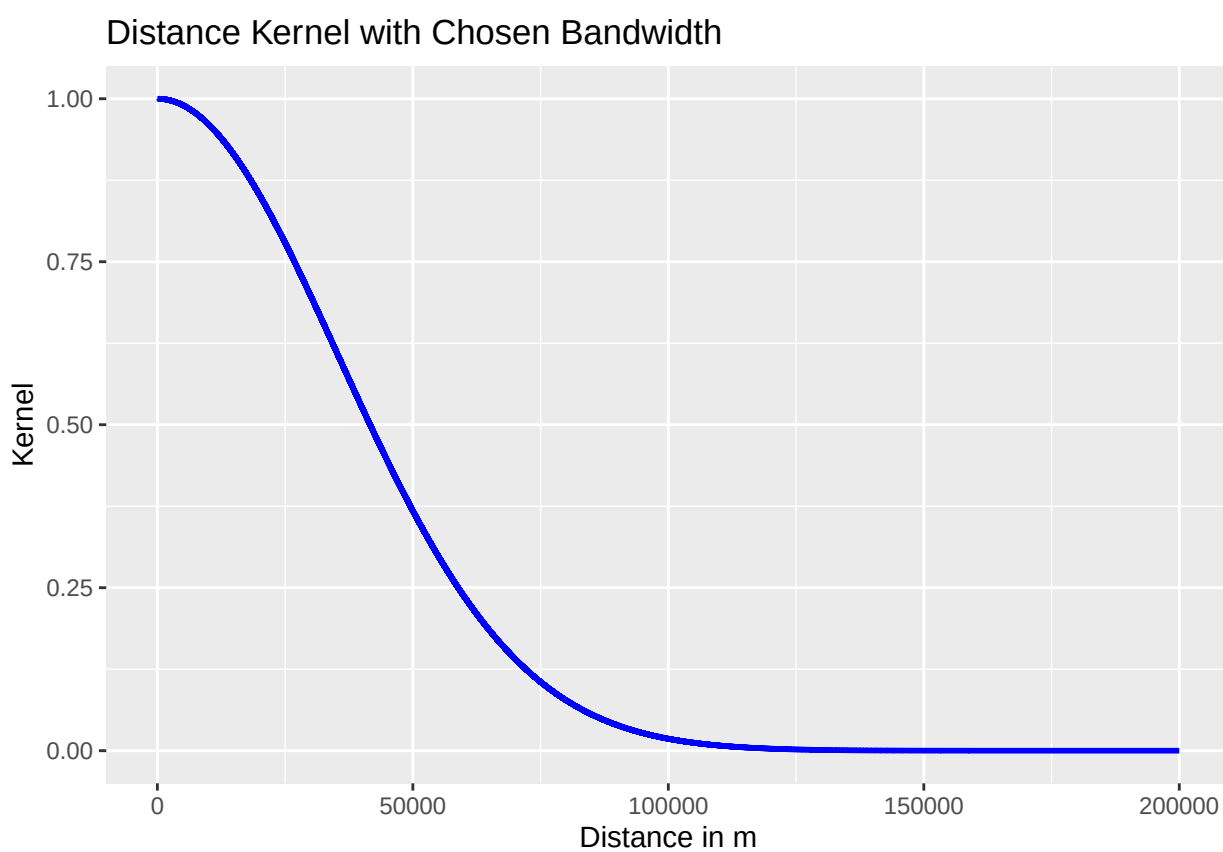
For this purpose, use the function distHaversine from the R package geosphere. The second to account for the distance between the day a temperature measurement was made and the day of interest. The third to account for the distance between the hour of the day a temperature measurement was made and the hour of interest. Choose an appropriate smoothing coefficient or width for each of the three kernels above. No cross-validation should be used. Instead, choose manually a width that gives large kernel values to closer points and small values to distant points. Show this with a plot of the kernel value as a function of distance. Help: Note that the file temps50k.csv may contain temperature measurements that are posterior to the day and hour of your forecast. You must filter such measurements out, i.e. they cannot be used to compute the forecast.

Finally, repeat the exercise above by combining the three kernels into one by multiplying them, instead of summing them up. Compare the results obtained in both cases and elaborate on why they may differ.The only R package that is allowed to solve the assignment 1 is the geosphere package(specifically, the function distHaversine).

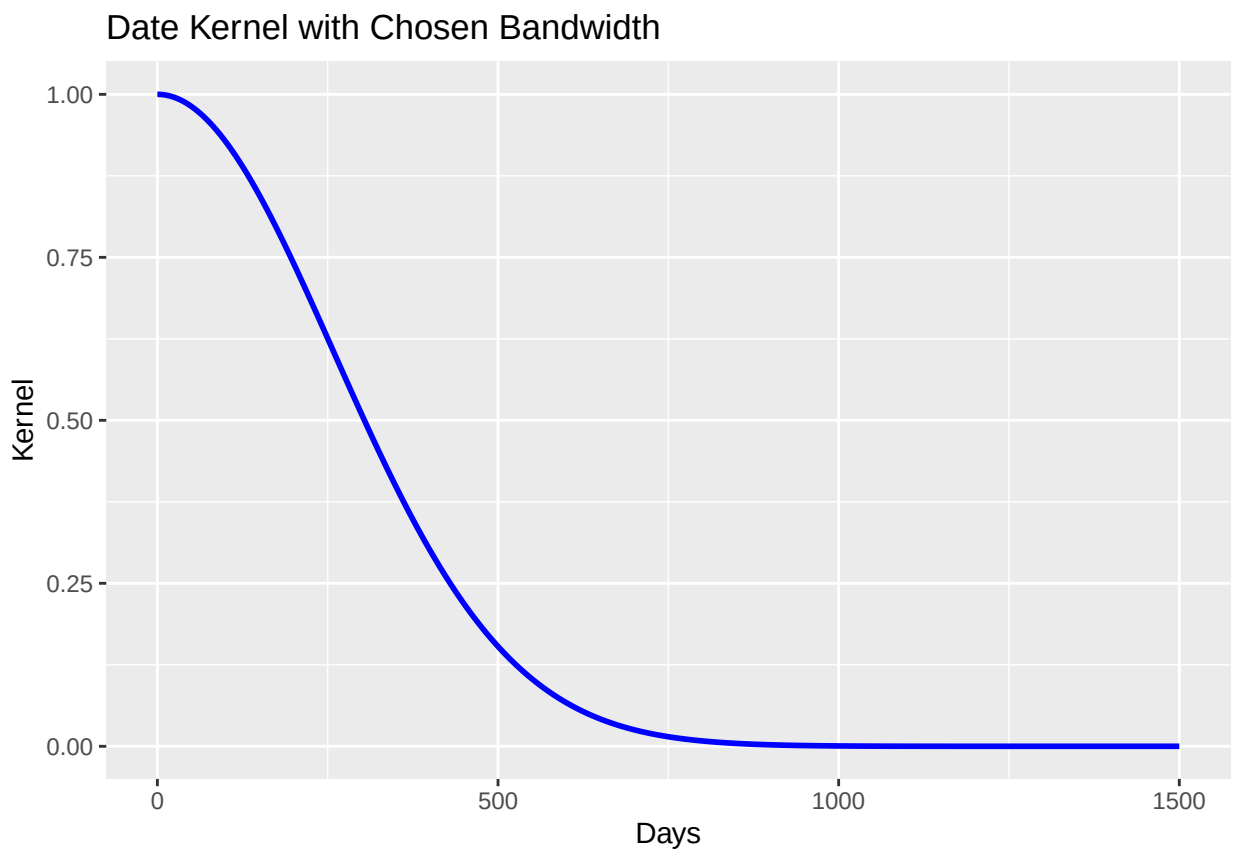
```
## The legacy packages maptools, rgdal, and rgeos, underpinning the sp package,
## which was just loaded, will retire in October 2023.
## Please refer to R-spatial evolution reports for details, especially
## https://r-spatial.org/r/2023/05/15/evolution4.html.
## It may be desirable to make the sf package available;
## package maintainers should consider adding sf to Suggests:.
## The sp package is now running under evolution status 2
##      (status 2 uses the sf package in place of rgdal)
```

```r
set.seed(1234567890)

# Merge data
st <- merge(stations, temps, by = "station_number")
# Ensure longitude and latitude are within valid ranges
stations$longitude <- ifelse(stations$longitude > 180, stations$longitude - 360, stations$longitude)
stations$latitude <- ifelse(stations$latitude > 90, 90 - (stations$latitude - 90), stations$latitude)

st$longitude <- ifelse(st$longitude > 180, st$longitude - 360, st$longitude)
st$latitude <- ifelse(st$latitude > 90, 90 - (st$latitude - 90), st$latitude)


# Selecting a specific station as the point of focus
set.seed(2023)
chosen_station_id <- 84260  #Orberga
chosen_station_location <-
  stations[stations$station_number == chosen_station_id, c("longitude", "latitude")]
chosen_station_data <- st[st$station_number == chosen_station_id, ]
analysis_date <- "2000-02-25"

# Generating a sequence of hours for the analysis day
analysis_hours <- seq(as.POSIXct("2000-02-25 04:00"), as.POSIXct("2000-02-25 24:00"), by = "2 hour")
formatted_hours <- format(analysis_hours, format = "%H:%M:%S")

# Calculating the maximum distance from the chosen station
maximum_distance <- max(distHaversine(chosen_station_location, st[, c("longitude", "latitude")]))

# Determining the maximum difference in dates
maximum_date_difference <- max(as.vector(difftime(as.Date(analysis_date),
                                                  as.Date(st$date), units = "days")))

# Manually chosen widths for each type kernel
h_distance_chosen <- 50000
h_date_chosen <- 365
h_time_chosen <- 3

# Redefine the scales with the chosen widths
distance_scale_chosen <- seq(0, 200000, 1) / h_distance_chosen
date_scale_chosen <- seq(0, 1500, 1) / h_date_chosen
time_scale_chosen <- seq(-12, 12, 1) / h_time_chosen

# Recalculating kernels with chosen widths
kernel_distance_chosen <- exp(-(distance_scale_chosen)^2)
kernel_date_chosen <- exp(-(date_scale_chosen)^2)
kernel_time_chosen <- exp(-(time_scale_chosen)^2)

# Plotting the kernels with chosen bandwidths
distance_plot_chosen <- ggplot(data.frame(kernel_distance_chosen, seq(0, 200000, 1)),
                               aes(x = seq(0, 200000, 1), y = kernel_distance_chosen)) +
  geom_line(color = "blue", linewidth = 1) +
  ggtitle("Distance Kernel with Chosen Bandwidth") +
  xlab("Distance in m") +
  ylab("Kernel")
```

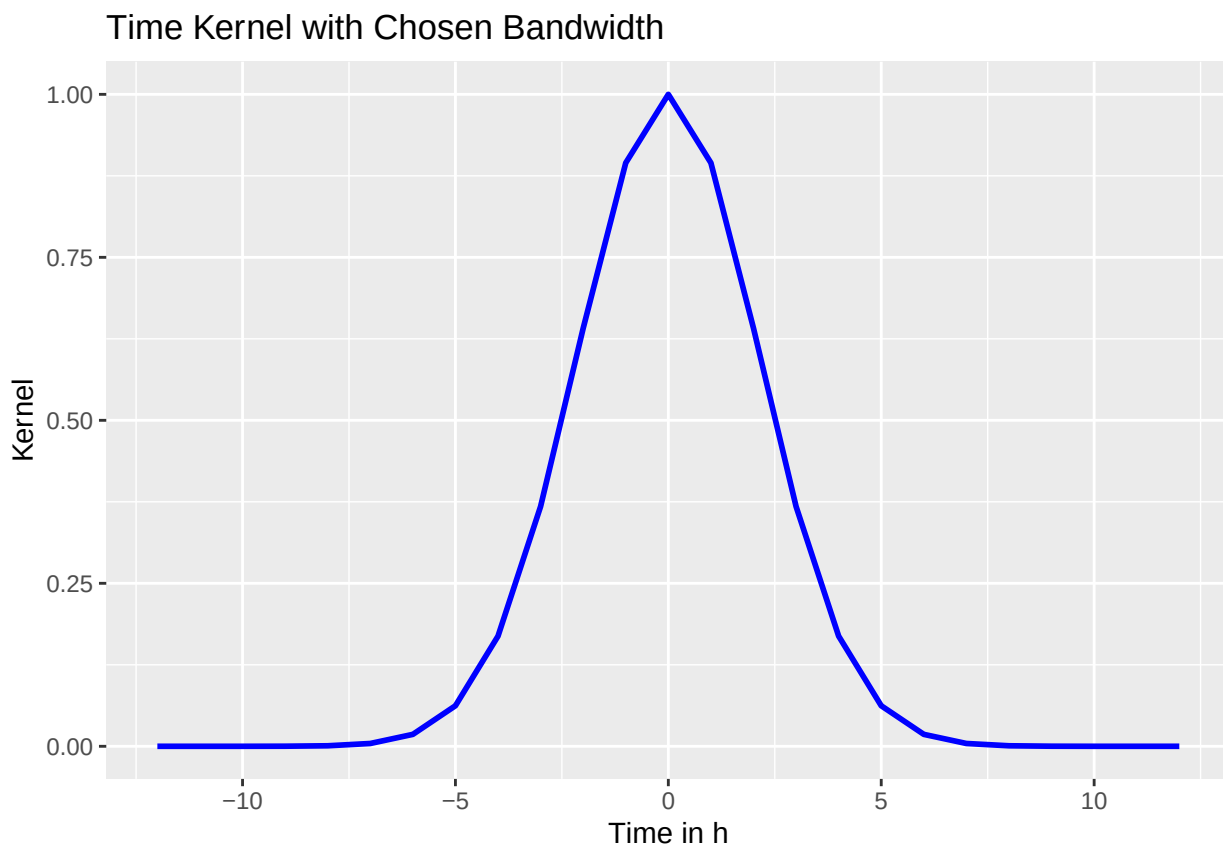
```r
date_plot_chosen <- ggplot(data.frame(kernel_date_chosen, seq(0, 1500, 1)), aes(x = seq(0, 1500, 1),
                                                                                 y = kernel_date_chosen)
  geom_line(color = "blue", linewidth = 1) +
  ggtitle("Date Kernel with Chosen Bandwidth") +
  xlab("Days") +
  ylab("Kernel")

time_plot_chosen <- ggplot(data.frame(kernel_time_chosen, seq(-12, 12, 1)), aes(x = seq(-12, 12, 1),
                                                                                 y = kernel_time_chosen)
  geom_line(color = "blue", linewidth = 1) +
  ggtitle("Time Kernel with Chosen Bandwidth") +
  xlab("Time in h") +
  ylab("Kernel")

# Display the plots with chosen bandwidths
distance_plot_chosen
```




```r
date_plot_chosen
```




```
time_plot_chosen
```

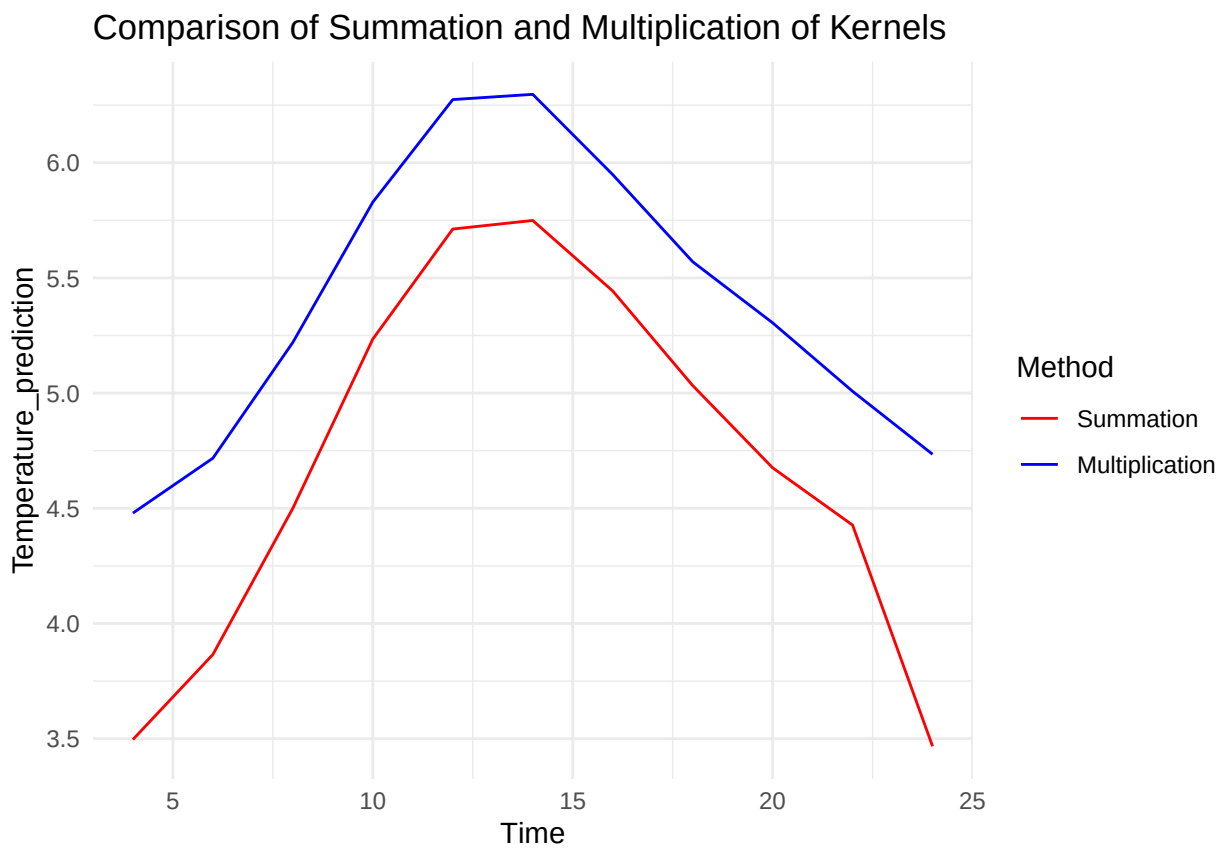


For the distance-based part of the model, a 50000 meters range was selected as the key distance. This means that weather stations closer than 50 km to the target location are given more importance in the predictions. On the other hand, stations that are more than 100 km away have very little effect on the forecast. This approach is based on the idea that weather can change a lot over long distances. So, the model mostly uses data from nearby stations, as shown in the plot 1, where stations far away don't contribute much to the predictions. The width of the days is set to 365 (a year) and the time is set to 3 hours because it is more likely that we will have the correct predictions if the time is close to the intitial.

```
#all scales should have the same length
max_length <- max(length(distance_scale_chosen), length(date_scale_chosen), length(time_scale_chosen))

#extend the shorter scales to the length of the longest one
distance_scale_extended <- seq(0, max(length(distance_scale_chosen), max_length - 1),
                               length.out = max_length)
date_scale_extended <- seq(0, max(length(date_scale_chosen), max_length - 1),
                           length.out = max_length)
time_scale_extended <- seq(-12, 12,
                           length.out = max_length)

#recalculate the kernels with extended scales
kernel_distance_extended <- exp(-(distance_scale_extended / h_distance_chosen)^2)
kernel_date_extended <- exp(-(date_scale_extended / h_date_chosen)^2)
kernel_time_extended <- exp(-(time_scale_extended / h_time_chosen)^2)

#filter out future measurements
forecast_date_time <- as.POSIXct(paste(analysis_date, "24:00", sep = " "))
```

```r
st <- subset(st, as.POSIXct(paste(st$date, st$time)) <= forecast_date_time)

#define the point and time of interest for the prediction
point_of_interest <- c(b = chosen_station_location$longitude, a = chosen_station_location$latitude)
time_of_interest <- formatted_hours

#function to calculate Gaussian kernel
gaussian_kernel  <- function(diff,bandwidth){
  exp(-0.5 * diff^2/bandwidth^2)
}

#sum
K1 <- function(diffs, bandwidths){
  gaussian_kernel(diffs[1],bandwidths[1]) +
    gaussian_kernel(diffs[2],bandwidths[2]) +
    gaussian_kernel(diffs[3],bandwidths[3])
}

#prod
K2 <- function(diffs, bandwidths){
  gaussian_kernel(diffs[1],bandwidths[1]) *
    gaussian_kernel(diffs[2],bandwidths[2]) *
    gaussian_kernel(diffs[3],bandwidths[3])
}

#initialize vectors to store the results
predictions_sum <- numeric(length(time_of_interest))
predictions_product <- numeric(length(time_of_interest))

#loop over each time point to calculate predictions
for (i in 1:length(time_of_interest)) {
  # Calculate distance, date, and time differences
  distance_diff <- distHaversine(point_of_interest, st[, c("longitude", "latitude")])
  date_diff <- as.numeric(as.Date(analysis_date) - as.Date(st$date))
  time_diff <- abs(as.numeric(difftime(strptime(time_of_interest[i], format = "%H:%M:%S"),
                                       strptime(st$time, format = "%H:%M:%S"), units = "hours")))

  denominator <- sum(sapply(1:nrow(st), function(x){
    K1(diffs = c(distance_diff[x], date_diff[x], time_diff[x]),
       bandwidths = c(h_distance_chosen, h_date_chosen, h_time_chosen)) * st$air_temperature[x]
  }))

  numerator <- sum(sapply(1:nrow(st), function(x){
    K1(diffs = c(distance_diff[x], date_diff[x], time_diff[x]),
       bandwidths = c(h_distance_chosen, h_date_chosen, h_time_chosen))
  }))

  predictions_sum[i] <- denominator/numerator

  denominator <- sum(sapply(1:nrow(st), function(x){
    K2(diffs = c(distance_diff[x], date_diff[x], time_diff[x]),
       bandwidths = c(h_distance_chosen, h_date_chosen, h_time_chosen)) * st$air_temperature[x]
  }))
```

```r
  numerator <- sum(sapply(1:nrow(st), function(x){
    K2(diffs = c(distance_diff[x], date_diff[x], time_diff[x]),
       bandwidths = c(h_distance_chosen, h_date_chosen, h_time_chosen))
  }))

  predictions_product[i] <- denominator/numerator
}

#create a dataframe for plotting the predictions
results_df <- data.frame(
  Time = seq(4,24,by=2),
  Summation = predictions_sum,
  Multiplication = predictions_product
)

#reshape the data for plotting with ggplot2
library(reshape2)
results_melted <- melt(results_df, id.vars = 'Time')

#predictions
ggplot(results_melted, aes(x = Time, y = value, color = variable)) +
  geom_line() +
  scale_color_manual(values = c('Summation' = 'red', 'Multiplication' = 'blue')) +
  labs(title = "Comparison of Summation and Multiplication of Kernels",
       x = "Time",
       y = "Temperature_prediction",
       color = "Method") +
  theme_minimal()
```



The chosen station is Orberga, 84260 (id). The prediction date is 2000-02-25 and the temperature was 5.2 at 00:00:00. The models are close to the result with the multiplication model to seem to be closer.

The multiplication kernel has higher prediction results compared to the sum model.

When we sum the kernels, we are effectively saying that each feature (distance, time, and date) independently contributes to the temperature prediction. If any of the features is very similar to the target (for ex. the time is exactly what we want to predict), it will have a strong positive influence on the prediction, even if the other features are less similar.

When we multiply the kernels, all features must be similar to the target for the combined kernel to be high. If any feature is dissimilar (for ex. the date is very different from what we are interested in), it will reduce the combined kernel value.

2. Support Vector Machines

The code in the file Lab3Block1 2021 SVMs St.R performs SVM model selection to classify the spam dataset. To do so, the code uses the function ksvm from the R package kernlab, which also includes the spam dataset. All the SVM models to select from use the radial basis function kernel (also known as Gaussian) with a width of 0.05. The C parameter varies between the models. Run the code in the file Lab3Block1 2021 SVMs St.R and answer the following questions:

2.1 Which filter do you return to the user ? filter0, filter1, filter2 or filter3? Why?

	C	Trained on	Tested on
filter	*varying, optimized*	Training Data	Validation Data
filter0	min C from filter optim	Training Data	Validation Data
filter1	min C from filter optim	Training Data	Test Data
filter2	min C from filter optim	Training Data + Validation Data	Test Data
filter3	min C from filter optim	Training Data + Validation Data + Test Data	Test Data

```
##
## Attaching package: 'kernlab'

## The following object is masked from 'package:ggplot2':
##
##     alpha

## [1] "err0:"

## [1] 0.0675

## [1] "err1:"

## [1] 0.08489388

## [1] "err2:"

## [1] 0.082397

## [1] "err3:"

## [1] 0.02122347
```

As displayed in the first row of the table, we first use the validation data to optimize our C-value as a hyperparameter. We do not want to use the test data for this because we do not want a C-value overfit on the test data which we will use for a final error estimation.

In the hyperparameter optimization, the validation data plays the role of the "unseen" data to test for the best C value when a trained model should predict new data.

When the best C-value is chosen and locked, the test data will take the role of the "unseen" data to estimate our final error value for the model. This means we can add our validation data and training data together to train our final model using the locked-in hyperparameter. This gives the model training more data to train on.

From the error estimations:

err0 is disqualified as it tests our final error value on the validation data, which is the same we use to optimize our hyperparameter.

err3 is disqualified as the test set is included in the training set. This eliminates the advantages of having an "unseen" data to test on. Also, the model becomes likely overfit on the test data.

Between err1 and err2, the latter gives us a slightly lower (better) test error. This is likely because it also uses the validation partition to train on, which we have allowed according to the reasoning above.

However, we only need this partition to estimate the generalization error. For the sake of the user having a robust model to use in the future, we should return such a well trained model we can, using all of our data. Hence, filter3.

The answer to the question: We chose to filter3 to the user.

2.2. What is the estimate of the generalization error of the filter returned to the user? err0, err1, err2, or err3? Why?

According to the reasoning above, the generalization error is the error tested on unseen data. Hence we need to use err2, which is equal to 0.082397.

2.3. Prediction

Once a SVM has been fitted to the training data, a new point is essentially classified according to the sign of a linear combination of the kernel function values between the support vectors and the new point. You are asked to implement this linear combination for filter3. You should make use of the functions alphaindex, coef and b that return the indexes of the support vectors, the linear coefficients for the support vectors, and the negative intercept of the linear combination. See the help file of the kernlab package for more information. You can check if your results are correct by comparing them with the output of the function predict where you set type = "decision". Do so for the first 10 points in the spam dataset. Feel free to use the template provided in the Lab3Block1 2021 SVMs St.R file.

The formula for predicting new values is:

$$\hat{y}(x_*) = \hat{\alpha}^T K(X, x_*)$$

wheras $\hat{\alpha}$ are the linear coefficients for the support vectors estimated by the algorithm. K is the kernel function which in our case is the Gaussian RBF kernel $k(x, x_*) = exp(-\sigma||x - x_*||^2)$. When predicting new values using SVM, X will be our support vectors and x_* will be the input values of the observations we are predicting. $\hat{\alpha}_0$ is a intercept and a bias term that can be added to the final result.

```
# The Gaussian RBF kernel
K <- function(x1,x2,sigma){
  # ||x-x_*||
  distance <- dist(rbind(x1,x2))
  exp(-sigma*distance^2)
}

# The indexes of the support vectors identified in our spam-dataset
sv<-alphaindex(filter3)[[1]]
# The linear coefficients to the corresponding support vectors
co<-coef(filter3)[[1]]
# The intercept/bias term
inte<- - b(filter3)


k<-NULL
for(i in 1:10){ # We produce predictions for just the first 10 points in the dataset.
  k2<-NULL
  for(j in 1:length(sv)){
    # Saving each support vector combination for later summarize
    # co[j] = alpha_hat_j
```

```r
    k2<- c(k2,
           co[j] * K(unlist(spam[i,-58]),unlist(spam[sv[j],-58]), sigma = 0.05))
  }
  # Summarizing every support vector combination and adding the intercept
  k<-c(k, sum(k2,inte))
}




data.frame(manual_pred = k, function_pred = predict(filter3,spam[1:10,-58], type = "decision"))
```

```
##    manual_pred function_pred
## 1    -1.998999     -1.998999
## 2     1.560584      1.560584
## 3     1.000278      1.000278
## 4    -1.756815     -1.756815
## 5    -2.669577     -2.669577
## 6     1.291312      1.291312
## 7    -1.068444     -1.068444
## 8    -1.312493     -1.312493
## 9     1.000184      1.000184
## 10   -2.208639     -2.208639
```

Hence, the manual prediction matches the function prediction output.

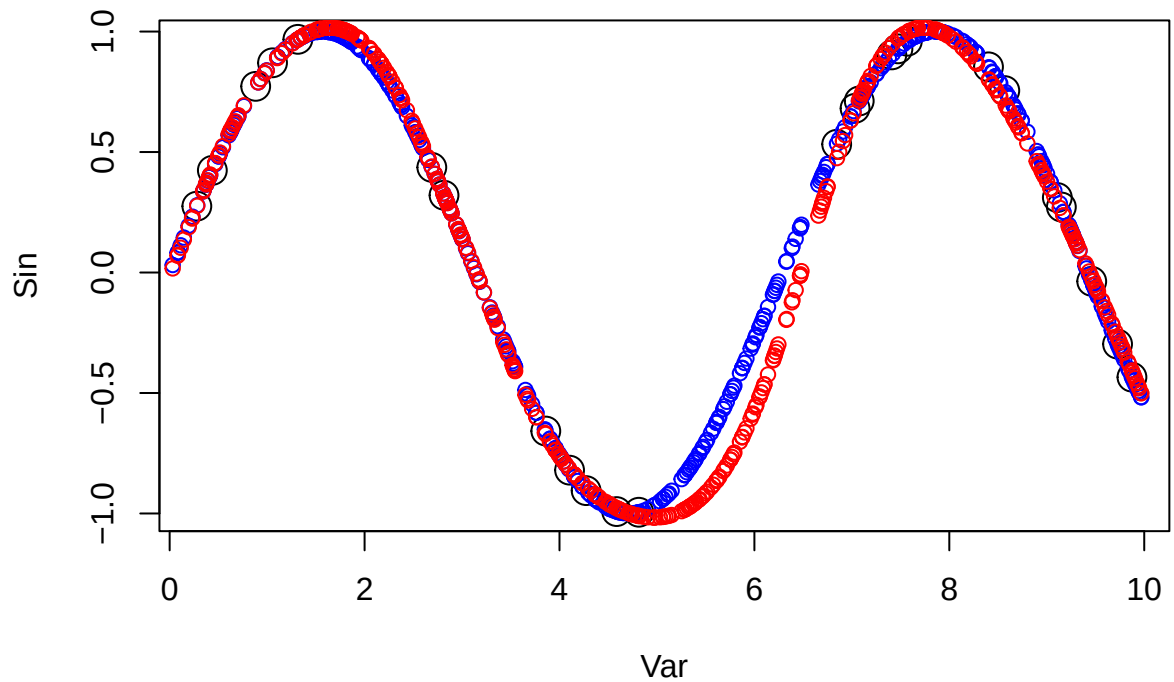
Question 3

3.1

Train a neural network to learn the trigonometric sine function. To do so, sample 500 points uniformly at random in the interval [0,10]. Apply the sine function to each point. The resulting value pairs are the data points available to you. Use 25 of the 500 points for training and the rest for test. Use one hidden layer with 10 hidden units. You do not need to apply early stopping. Plot the training and test data, and the predictions of the learned NN on the test data. You should get good results. Comment your results.

From the plot; Black points represent the training data. We use these points to train the neural networks.(actual data points of the sine function). Train points looks well distributed from 0 to 10 'var' values. Blue points are the test points and the red points are the points that neural network predict.

The plot that we have has a good fit of the neural network to the sine function. We can see that neural network prediction line fallows the test line very well. This shows that the neural network successfully learned to approximate the sine function with the data that we have for this part of the question.



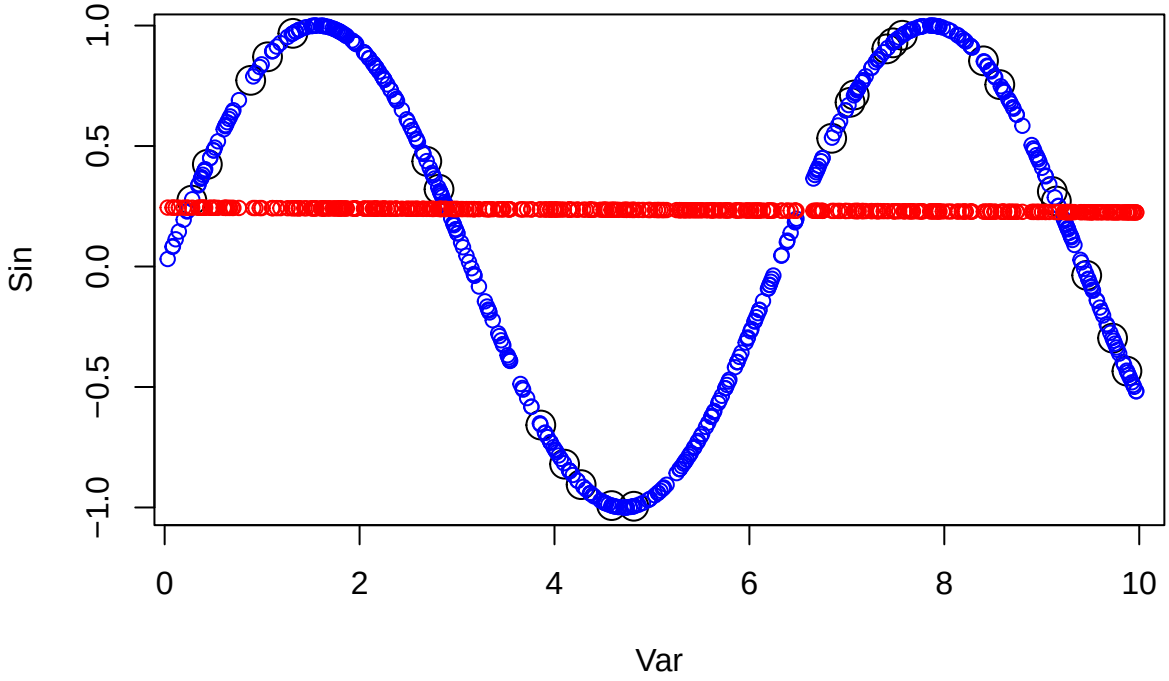
3.2

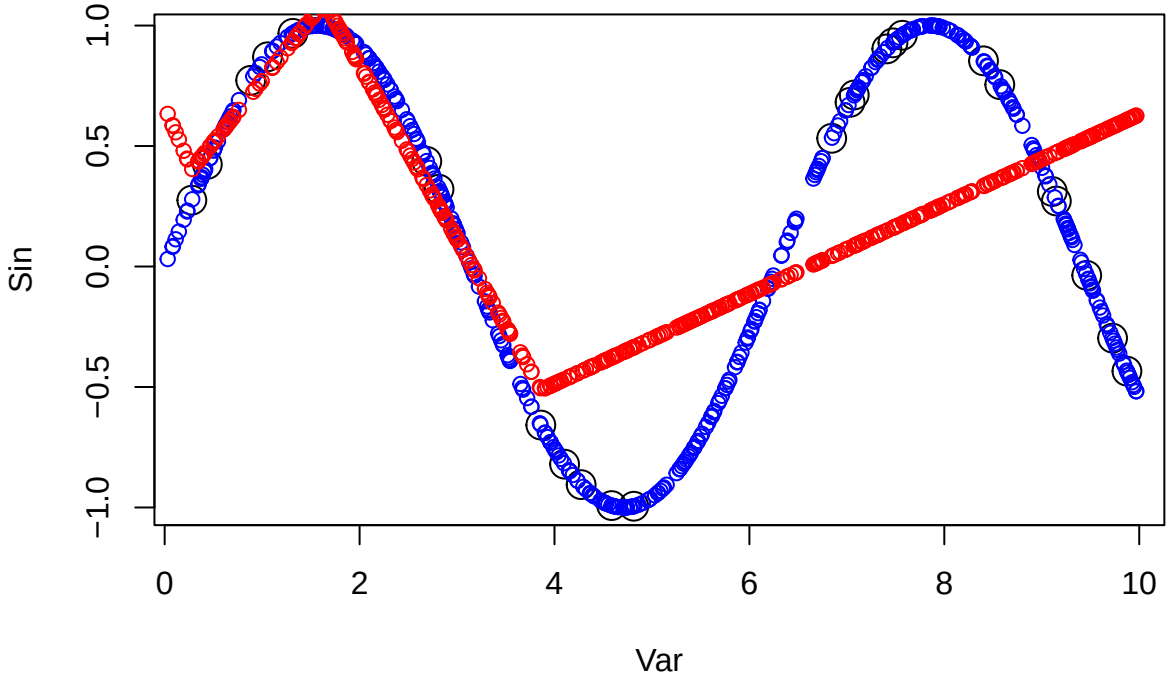
In question (1), you used the default logistic (a.k.a. sigmoid) activation function, i.e. act.fct = "logistic". Repeat question (1) with the following custom activation functions: h1 (x) = x, h2 (x) = max{0, x} and h3 (x) = ln(1 + exp x) (a.k.a. linear, ReLU and softplus). See the help file of the neuralnet package to learn how to use custom activation functions. Plot and comment your results.

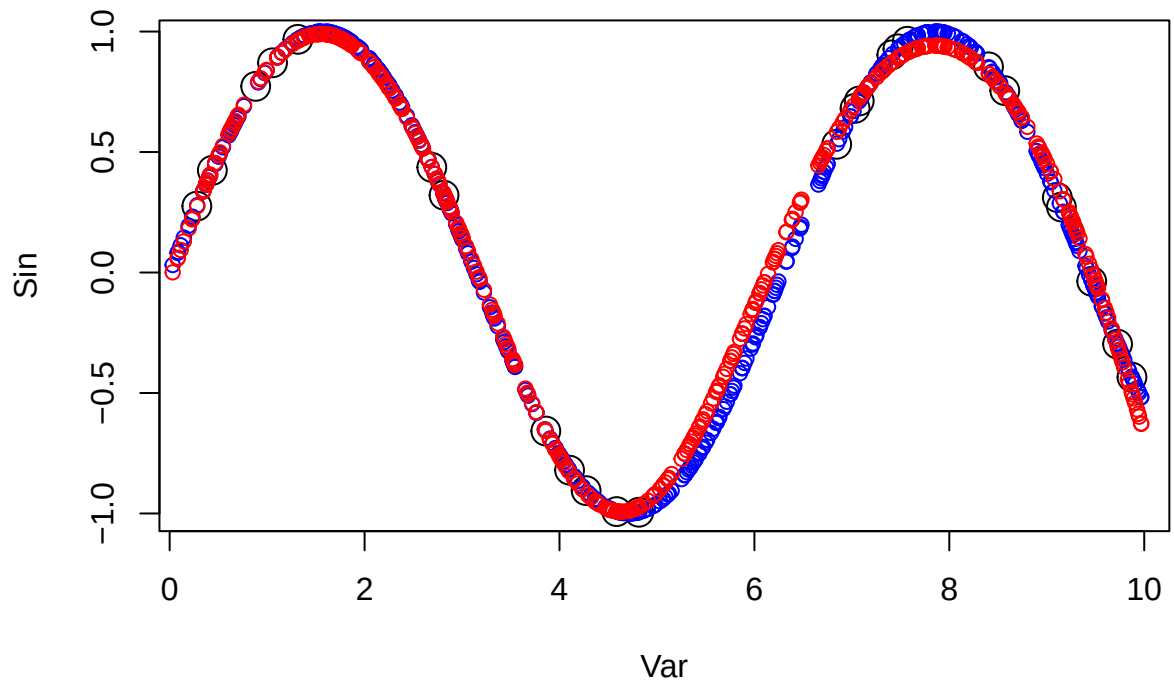
1) h1 (x)=x; the predictions that are made by the neural network are plotted red and test points are in blue. There is a deviation from the blue points, shows that the model is not capturing the sine function well. Activation function h1 is a linear functiona and 'sine' is non linear. With h1 being liear it is expected to have difficulties capturing the properties of a non-linear function. To be able to get better results for this non-linear functions like sigmoid can be used.

2) h2(x)= max{0, x}; as we can see from the plots using Relu activation function made a change in the nn predictions. Red dots which represent neural network predictions ave a non-linear behavior for h2 function. But, predictions are still do not match the blue data points(test points). We can say that our prediction model improved but still not perfectly fitting the test points. To .mprove h2 hidden layer units can be improved.

3) h3 (x) = ln(1 + exp x); The red points are matching the blue test points. This shows better performance then the other two functions(h1 and h2) that we use.


1.0
0.5
0.0
−0.5
−1.0
Sin
0
2
4
6
8
10
Var

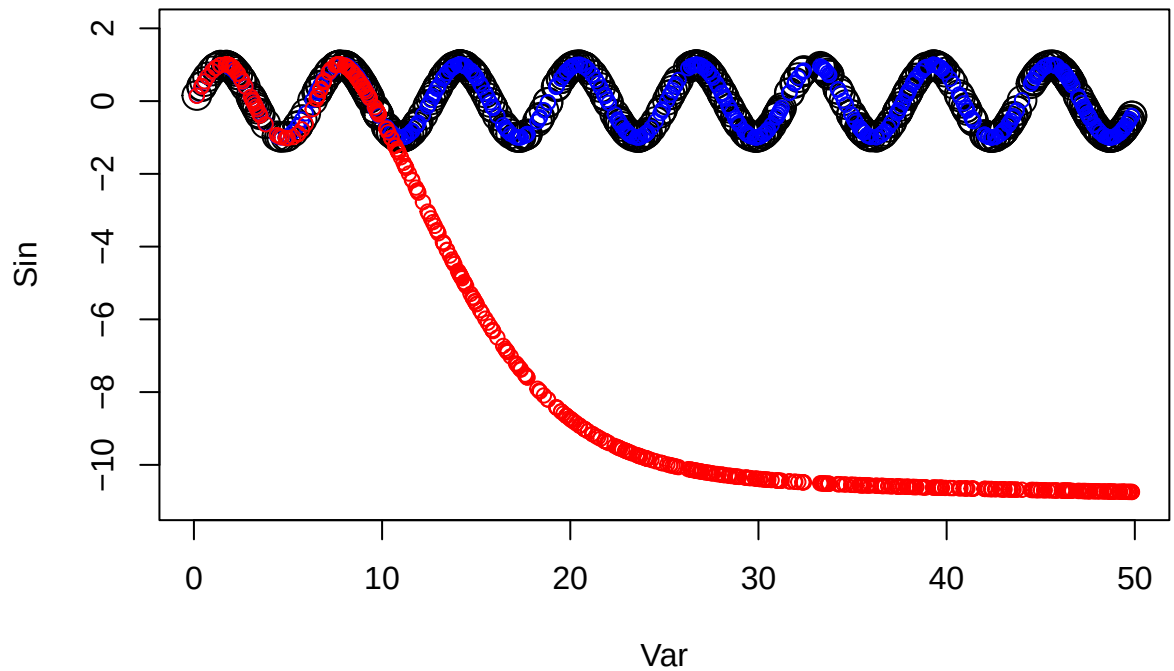

Sin
1.0
0.5
0.0
−0.5
−1.0
0
2
4
6
8
10
Var



3.3

Sample 500 points uniformly at random in the interval [0,50], and apply the sine function to each point. Use the NN learned in question (1) to predict the sine function value for these new 500 points. You should get mixed results. Plot and comment your results.

For this question we are using the NN from question 1. From the plot between 0-10 Var values NN points fits the test points. Which is something we can expect, because initialized model in question 1 is trained with values from 0 to 10(so this part is familiar ith the structure). After 'var'= 10, nn predicitions doesn't fit the test points.



3.4

In question (3), the predictions seem to converge to some value. Explain why this happens. To answer this question, you may need to get access to the weights of the NN learned. You can do it by running nn or nn$weights where nn is the NN learned.

When we calculate "W1*X+b1" where X is a large scalar. In our W1 and W2 we have positive and negative weight values. After the multiplication with large scalar, resulting matrix will have positive and negative large values. Multiplying with the weights makes the result larger. Applying sigmoid function to W1X+b1. when the resulting values are large negative numbers sigmoid will transform them to zeros and if the resulting numbers are large positive numbers they will be transformed closer to one.

q-> is a matrix with 10 elements and 1 column 10 rows.(1,0,1,0,0,1,0,0,0,0)

yˆ=W2*q+b2

yˆ=(0.7716196-16.0529801+4.3076898)+(-0.1088032) yˆ=-10.9736707-0.1088032 yˆ=-11.0824

This type of convergence is often related to the saturation of the activation function used within the neural network.

weights; The specific values of the weights and biases can cause the outputs to converge to a certain value when inputs are large. if the input values are very large, as in our calculation above, the sigmoid neurons may become saturated. This can cause them to output extreme values (close to 0 or 1), which can lead to bad performance for inputs that fall outside the range the network was trained on.

Also if the network has overfitted to the training data, it may not generalize well to the new data. This can lead to a bad prediction for the neural network.

source: Book-Machine Learning A First Course for Engineers and Scientists, Lindholm A., Walhström N., Lindsten F., Schön B. T., 2022.

Table 2: Bias 1

-11.8708310
-0.9153178
-3.0316259
1.5313105
6.5554370
-11.7685250
1.3973250
0.2032402
0.2948804
-0.2132649

Table 3: Weights 1

4.0424940
-0.5368677
0.2603355
-0.5487656
-2.2338390
1.7878620
-1.2598340
-2.1974036
-0.5886005
-0.0317728

Table 4: Bias 2

-0.1088032

Table 5: Weights 2

0.7716196
-1.0436754
-16.0529801
-1.2718986
3.2805756
4.3076898
0.1220703
-1.2234185
-2.7618205
2.3696931

```r
library(sigmoid)
```

```
## Warning: package 'sigmoid' was built under R version 4.3.2
```

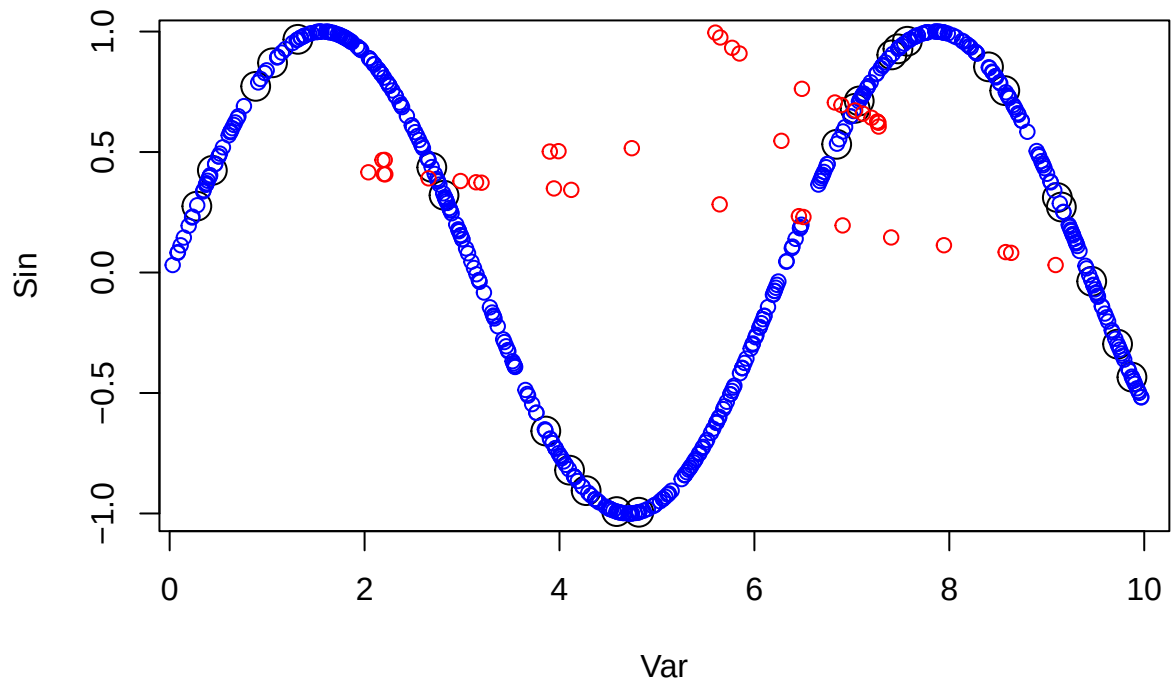
```r
sigmoid<-function(x){
  return(1/(1+ exp(-x)))
}

#extract weights and bias
W1 <- nn$weights[[1]][[1]][2, ]
b1 <- nn$weights[[1]][[1]][1, ]
W2 <- nn$weights[[1]][[2]][2:11, ]
b2 <- nn$weights[[1]][[2]][1, ]

q <- sigmoid(W1 * 100 + b1)

cat("value converged to(sigmoid): ", W2%*%q+b2)
```

```
## value converged to(sigmoid):  -11.00524
```

3.5

Sample 500 points uniformly at random in the interval [0,10], and apply the sine function to each point. Use all these points as training points for learning a NN that tries to predict x from sin(x), i.e. unlike before when the goal was to predict sin(x) from x. Use the learned NN to predict the training data. You should get bad results. Plot and comment your results. Help: Some people get a convergence error in this question. It can be solved by stopping the training before reaching convergence by setting threshold = 0.1.

This time, 'var' is predicted from 'sin' other way around. Red points which are the points for the neural network prediction doesn't fit with the test points. Model is not predicting the 'Var' values properly. Sine function can return the same result for several values. Because of this neural network can get confused while predicting. For this kind of inverse prediction tasks(more complex tasks) another model can be used. More additional features can be added for seperating different Var values for the same Sin value.



Appendix

Chunk Label: setup

```
knitr::opts_chunk$set(echo = TRUE)
```

Chunk Label: 1_0

```
set.seed(1234567890)
library(geosphere)
library(ggplot2)
```

Chunk Label: 1_data

```
# Read the data
stations <- read.csv("C:/Users/maxib/Downloads/stations.csv", fileEncoding = "latin1")
temps <- read.csv("C:/Users/maxib/Downloads/temps50k.csv")
```

Chunk Label: 1_1

```r
set.seed(1234567890)

# Merge data
st <- merge(stations, temps, by = "station_number")
# Ensure longitude and latitude are within valid ranges
stations$longitude <- ifelse(stations$longitude > 180, stations$longitude - 360, stations$longitude)
stations$latitude <- ifelse(stations$latitude > 90, 90 - (stations$latitude - 90), stations$latitude)

st$longitude <- ifelse(st$longitude > 180, st$longitude - 360, st$longitude)
st$latitude <- ifelse(st$latitude > 90, 90 - (st$latitude - 90), st$latitude)


# Selecting a specific station as the point of focus
set.seed(2023)
chosen_station_id <- 84260  #Orberga
chosen_station_location <-
  stations[stations$station_number == chosen_station_id, c("longitude", "latitude")]
chosen_station_data <- st[st$station_number == chosen_station_id, ]
analysis_date <- "2000-02-25"

# Generating a sequence of hours for the analysis day
analysis_hours <- seq(as.POSIXct("2000-02-25 04:00"), as.POSIXct("2000-02-25 24:00"), by = "2 hour")
formatted_hours <- format(analysis_hours, format = "%H:%M:%S")

# Calculating the maximum distance from the chosen station
maximum_distance <- max(distHaversine(chosen_station_location, st[, c("longitude", "latitude")]))

# Determining the maximum difference in dates
maximum_date_difference <- max(as.vector(difftime(as.Date(analysis_date),
                                                  as.Date(st$date), units = "days")))

# Manually chosen widths for each type kernel
h_distance_chosen <- 50000
h_date_chosen <- 365
h_time_chosen <- 3

# Redefine the scales with the chosen widths
distance_scale_chosen <- seq(0, 200000, 1) / h_distance_chosen
date_scale_chosen <- seq(0, 1500, 1) / h_date_chosen
time_scale_chosen <- seq(-12, 12, 1) / h_time_chosen

# Recalculating kernels with chosen widths
kernel_distance_chosen <- exp(-(distance_scale_chosen)^2)
kernel_date_chosen <- exp(-(date_scale_chosen)^2)
kernel_time_chosen <- exp(-(time_scale_chosen)^2)

# Plotting the kernels with chosen bandwidths
distance_plot_chosen <- ggplot(data.frame(kernel_distance_chosen, seq(0, 200000, 1)),
                               aes(x = seq(0, 200000, 1), y = kernel_distance_chosen)) +
  geom_line(color = "blue", linewidth = 1) +
  ggtitle("Distance Kernel with Chosen Bandwidth") +
```

```
  xlab("Distance in m") +
  ylab("Kernel")

date_plot_chosen <- ggplot(data.frame(kernel_date_chosen, seq(0, 1500, 1)), aes(x = seq(0, 1500, 1),
                                                                                 y = kernel_date_chosen))
  geom_line(color = "blue", linewidth = 1) +
  ggtitle("Date Kernel with Chosen Bandwidth") +
  xlab("Days") +
  ylab("Kernel")

time_plot_chosen <- ggplot(data.frame(kernel_time_chosen, seq(-12, 12, 1)), aes(x = seq(-12, 12, 1),
                                                                                 y = kernel_time_chosen))
  geom_line(color = "blue", linewidth = 1) +
  ggtitle("Time Kernel with Chosen Bandwidth") +
  xlab("Time in h") +
  ylab("Kernel")

# Display the plots with chosen bandwidths
distance_plot_chosen
date_plot_chosen
time_plot_chosen
```

Chunk Label: 1_2

```
#all scales should have the same length
max_length <- max(length(distance_scale_chosen), length(date_scale_chosen), length(time_scale_chosen))

#extend the shorter scales to the length of the longest one
distance_scale_extended <- seq(0, max(length(distance_scale_chosen), max_length - 1),
                               length.out = max_length)
date_scale_extended <- seq(0, max(length(date_scale_chosen), max_length - 1),
                           length.out = max_length)
time_scale_extended <- seq(-12, 12,
                           length.out = max_length)

#recalculate the kernels with extended scales
kernel_distance_extended <- exp(-(distance_scale_extended / h_distance_chosen)^2)
kernel_date_extended <- exp(-(date_scale_extended / h_date_chosen)^2)
kernel_time_extended <- exp(-(time_scale_extended / h_time_chosen)^2)

#filter out future measurements
forecast_date_time <- as.POSIXct(paste(analysis_date, "24:00", sep = " "))
st <- subset(st, as.POSIXct(paste(st$date, st$time)) <= forecast_date_time)

#define the point and time of interest for the prediction
point_of_interest <- c(b = chosen_station_location$longitude, a = chosen_station_location$latitude)
time_of_interest <- formatted_hours

#function to calculate Gaussian kernel
gaussian_kernel  <- function(diff,bandwidth){
  exp(-0.5 * diff^2/bandwidth^2)
}
```

```r
#sum
K1 <- function(diffs, bandwidths){
  gaussian_kernel(diffs[1],bandwidths[1]) +
    gaussian_kernel(diffs[2],bandwidths[2]) +
    gaussian_kernel(diffs[3],bandwidths[3])
}

#prod
K2 <- function(diffs, bandwidths){
  gaussian_kernel(diffs[1],bandwidths[1]) *
    gaussian_kernel(diffs[2],bandwidths[2]) *
    gaussian_kernel(diffs[3],bandwidths[3])
}

#initialize vectors to store the results
predictions_sum <- numeric(length(time_of_interest))
predictions_product <- numeric(length(time_of_interest))

#loop over each time point to calculate predictions
for (i in 1:length(time_of_interest)) {
  # Calculate distance, date, and time differences
  distance_diff <- distHaversine(point_of_interest, st[, c("longitude", "latitude")])
  date_diff <- as.numeric(as.Date(analysis_date) - as.Date(st$date))
  time_diff <- abs(as.numeric(difftime(strptime(time_of_interest[i], format = "%H:%M:%S"),
                                       strptime(st$time, format = "%H:%M:%S"), units = "hours")))

  denominator <- sum(sapply(1:nrow(st), function(x){
    K1(diffs = c(distance_diff[x], date_diff[x], time_diff[x]),
       bandwidths = c(h_distance_chosen, h_date_chosen, h_time_chosen)) * st$air_temperature[x]
  }))

  numerator <- sum(sapply(1:nrow(st), function(x){
    K1(diffs = c(distance_diff[x], date_diff[x], time_diff[x]),
       bandwidths = c(h_distance_chosen, h_date_chosen, h_time_chosen))
  }))

  predictions_sum[i] <- denominator/numerator

  denominator <- sum(sapply(1:nrow(st), function(x){
    K2(diffs = c(distance_diff[x], date_diff[x], time_diff[x]),
       bandwidths = c(h_distance_chosen, h_date_chosen, h_time_chosen)) * st$air_temperature[x]
  }))

  numerator <- sum(sapply(1:nrow(st), function(x){
    K2(diffs = c(distance_diff[x], date_diff[x], time_diff[x]),
       bandwidths = c(h_distance_chosen, h_date_chosen, h_time_chosen))
  }))

  predictions_product[i] <- denominator/numerator
}

#create a dataframe for plotting the predictions
results_df <- data.frame(
```

```
  Time = seq(4,24,by=2),
  Summation = predictions_sum,
  Multiplication = predictions_product
)

#reshape the data for plotting with ggplot2
library(reshape2)
results_melted <- melt(results_df, id.vars = 'Time')

#predictions
ggplot(results_melted, aes(x = Time, y = value, color = variable)) +
  geom_line() +
  scale_color_manual(values = c('Summation' = 'red', 'Multiplication' = 'blue')) +
  labs(title = "Comparison of Summation and Multiplication of Kernels",
       x = "Time",
       y = "Temperature_prediction",
       color = "Method") +
  theme_minimal()
```

Chunk Label: 2_1

```
library(kernlab)
set.seed(1234567890)

data(spam)
foo <- sample(nrow(spam))
spam <- spam[foo,]
spam[,-58]<-scale(spam[,-58])
tr <- spam[1:3000, ]
va <- spam[3001:3800, ]
trva <- spam[1:3800, ]
te <- spam[3801:4601, ]

by <- 0.3
err_va <- NULL
for(i in seq(by,5,by)){
  filter <- ksvm(type~.,data=tr,kernel="rbfdot",kpar=list(sigma=0.05),C=i,scaled=FALSE)
  mailtype <- predict(filter,va[,-58])
  t <- table(mailtype,va[,58])
  err_va <-c(err_va,(t[1,2]+t[2,1])/sum(t))
}

filter0 <- ksvm(type~.,data=tr,kernel="rbfdot",kpar=list(sigma=0.05),
                C=which.min(err_va)*by,scaled=FALSE)
mailtype <- predict(filter0,va[,-58])
t <- table(mailtype,va[,58])
err0 <- (t[1,2]+t[2,1])/sum(t)
print("err0:")
err0

filter1 <- ksvm(type~.,data=tr,kernel="rbfdot",kpar=list(sigma=0.05),
                C=which.min(err_va)*by,scaled=FALSE)
```

```r
mailtype <- predict(filter1,te[,-58])
t <- table(mailtype,te[,58])
err1 <- (t[1,2]+t[2,1])/sum(t)
print("err1:")
err1

filter2 <- ksvm(type~.,data=trva,kernel="rbfdot",kpar=list(sigma=0.05),
                C=which.min(err_va)*by,scaled=FALSE)
mailtype <- predict(filter2,te[,-58])
t <- table(mailtype,te[,58])
err2 <- (t[1,2]+t[2,1])/sum(t)
print("err2:")
err2

filter3 <- ksvm(type~.,data=spam,kernel="rbfdot",kpar=list(sigma=0.05),
                C=which.min(err_va)*by,scaled=FALSE)
mailtype <- predict(filter3,te[,-58])
t <- table(mailtype,te[,58])
err3 <- (t[1,2]+t[2,1])/sum(t)
print("err3:")
err3
```

Chunk Label: 2_3

```r
# The Gaussian RBF kernel
K <- function(x1,x2,sigma){
  # ||x-x_*||
  distance <- dist(rbind(x1,x2))
  exp(-sigma*distance^2)
}

# The indexes of the support vectors identified in our spam-dataset
sv<-alphaindex(filter3)[[1]]
# The linear coefficients to the corresponding support vectors
co<-coef(filter3)[[1]]
# The intercept/bias term
inte<- - b(filter3)


k<-NULL
for(i in 1:10){ # We produce predictions for just the first 10 points in the dataset.
  k2<-NULL
  for(j in 1:length(sv)){
    # Saving each support vector combination for later summarize
    # co[j] = alpha_hat_j
    k2<- c(k2,
           co[j] * K(unlist(spam[i,-58]),unlist(spam[sv[j],-58]), sigma = 0.05))
  }
  # Summarizing every support vector combination and adding the intercept
  k<-c(k, sum(k2,inte))
}
```

```r
data.frame(manual_pred = k, function_pred = predict(filter3,spam[1:10,-58], type = "decision"))
```

Chunk Label: Question 3.1

```r
library(neuralnet)

set.seed(1234567890)

Var <- runif(500, 0, 10)
mydata <- data.frame(Var, Sin = sin(Var))


tr <- mydata[1:25,]   # Training
te <- mydata[26:500,] # Test

# Random initialization of the weights in the interval [-1, 1]
winit <- runif(31, min = -1, max = 1)


nn <- neuralnet(Sin ~ Var, data = tr, hidden = 10, startweights = winit)

plot(tr, cex=2)
points(te, col = "blue", cex=1)
points(te[,1],predict(nn,te), col="red", cex1=1)
```

Chunk Label: Question 3.2

```r
h1<-function(x){
  x
}

nn_linear<-neuralnet(Sin ~ Var, data = tr, hidden = 10, act.fct = h1)

plot(tr, cex=2)
points(te, col = "blue", cex=1)
points(te[,1],predict(nn_linear,te), col="red", cex=1)

#ReLU-  h2 (x) = max{0, x}
#i get an error when i use pmax. so i did it in other way for h2.
#h2<-function(x) pmax(0,x)
h2 <- function(x) {
  ifelse(x > 0, x, 0)
}

nn_relu <- neuralnet(Sin ~ Var, data = tr,startweights = winit,
                     hidden = 10, act.fct = h2,threshold=0.15)
```

```r
plot(tr, cex=2)
points(te, col = "blue", cex=1)
points(te[,1],predict(nn_relu,te), col="red", cex=1)

#softplus
h3<-function(x) log(1+exp(x))

nn_softplus<-neuralnet(Sin~Var, data=tr, hidden=10, act.fct=h3)

plot(tr, cex=2)
points(te, col = "blue", cex=1)
points(te[,1],predict(nn_softplus,te), col="red", cex=1)
```

Chunk Label: Question 3.3

```r
set.seed(1234567890)

Var <- runif(500, 0, 50)
mydata_2 <- data.frame(Var, Sin = sin(Var))

pred_nn<-predict(nn,mydata_2)


plot(mydata_2, cex=2, ylim=c(-11,2))
points(mydata_2, col = "blue", cex=1)
points(mydata_2[,1],pred_nn, col="red", cex=1)
```

Chunk Label: question 3.4

```r
library(knitr)

plot(nn)

bias_1 <- matrix(
  c(-11.870831,
    -0.9153178,
    -3.0316259,
    1.5313105,
    6.555437,
    -11.768525,
    1.397325,
    0.2032402,
    0.2948804,
    -0.21326487
  ),
  nrow = 10, ncol = 1, byrow = TRUE
)
#colnames(bias_1) <- "bias_1"
#bias_1
```

```r
weight_1<-matrix(
  c(4.042494,
    -0.5368677,
    0.2603355,
    -0.5487656,
    -2.233839,
    1.787862,
    -1.259834,
    -2.1974036,
    -0.5886005,
    -0.03177278

  ),
  nrow = 10, ncol = 1, byrow = TRUE
)
#colnames(weight_1) <- "weights_1"
#weight_1


bias_2<-matrix(
  c( -0.1088032

  ),
    nrow = 1, ncol = 1, byrow = TRUE
)
#colnames(bias_2) <- "bias_2"
#bias_2

weights_2<-matrix(
  c( 0.7716196,
     -1.0436754,
     -16.0529801,
     -1.2718986,
     3.2805756,
     4.3076898,
     0.1220703,
     -1.2234185,
     -2.7618205,
    2.3696931
  ),
    nrow = 10, ncol = 1, byrow = TRUE
)
#colnames(weights_2) <- "weights_2"
#weights_2

kable(bias_1, caption = "Bias 1")
kable(weight_1, caption = "Weights 1")
kable(bias_2, caption = "Bias 2")
kable(weights_2, caption = "Weights 2")
```

Chunk Label: unnamed-chunk-1

```
library(sigmoid)
sigmoid<-function(x){
  return(1/(1+ exp(-x)))
}

#extract weights and bias
W1 <- nn$weights[[1]][[1]][2, ]
b1 <- nn$weights[[1]][[1]][1, ]
W2 <- nn$weights[[1]][[2]][2:11, ]
b2 <- nn$weights[[1]][[2]][1, ]

q <- sigmoid(W1 * 100 + b1)

cat("value converged to(sigmoid): ", W2%*%q+b2)
```

Chunk Label: Question 3.5

```
set.seed(1234567890)

Var <- runif(500, 0, 10)
mydata <- data.frame(Var, Sin = sin(Var))

tr <- mydata[1:25,]   # Training
te <- mydata[26:500,] # Test


nn5<-neuralnet(Var ~ Sin, data = tr,startweights = winit,
               hidden = 10,threshold=0.1)

plot(tr, cex=2)
points(te, col = "blue", cex=1)
points(predict(nn5,te),te[,1], col="red", cex=1)
```